

1 Inside the refactor

18 September 2026 · A guided tour from 27f644c to ff87e58

The nicest parts of this code are not the new filenames. They are the places where an important rule has become explicit: a tab has an identity independent of its position; an old save cannot approve a new close; a window cannot assume it owns the entire PDF memory budget; painting a button cannot itself run Git.

This is the expanded companion to the conversational walkthrough. It covers all ten commits after the pre-architecture baseline, including the repairs and the later five-step extraction plan. The endpoint is now committed: ff87e58 contains the preview cleanup, deduplication and icons/Settings split. The working tree was clean when this review began. There is no extra uncommitted code batch being quietly included in the story.

The tone is deliberately appreciative, but not promotional. “Enables later” means a plausible extension of a real boundary, not a shipped feature. “Tested” means the particular evidence described, not universal native correctness. This report complements `postmortem.typ` and `postpostmortem.typ`; it does not retract their conclusions about size, integration bugs or the still-large app.

Tour map

1	Inside the refactor	1
2	The campaign at a glance	2
2.1	What changed in each commit	2
2.2	What was already there	2
3	Documents: identity that survives movement	3
3.1	A tab is a record, not a slot	3
3.2	Saved bytes and visible edits have different identities	3
4	Saving: responsiveness without relaxing safety	3
4.1	Immutable handoff, then protected background IO	3
4.2	One resource lease, with the check inside it	4
4.3	Completion is a gate, not a notification	4
5	Tinymist: separate wire mechanics from document policy	5
5.1	A transport parser that knows nothing about the editor	5
5.2	Feature codecs borrow source instead of owning the world	5
5.3	Synchronization is a versioned plan; the app performs the IO	5
6	Preview: readiness is earned, resources are budgeted	6
6.1	A controller decides transitions, adapters execute them	6
6.2	An endpoint is not ready just because a message says so	6
6.3	PDF decoding is no longer a compiler responsibility	7
6.4	Keep the page map; rent the pixels	7
6.5	Multiple windows share a resource budget	7
7	Workspace and indexing: stop doing work nobody needs	8
7.1	Two workers, not one obsolete thread per request	8
7.2	Refusing new work must not destroy good old work	8
7.3	One workspace tree, many local Explorers	8
7.4	An index can admit what it does not know	9
8	Editing and views: make mutations deliberate	9
8.1	Completion is one consumable edit	9
8.2	The completion popup borrows its payload	9
8.3	Search selection and source navigation are different intents	10
8.4	Explorer painting returns requests, not side effects	10
9	Git: transactions below, actions above	10
9.1	Repository execution has its own home	10
9.2	Stage/unstage and panel mutations share a transaction boundary	10
9.3	Rendering a Fetch button cannot fetch	11
10	Windows, Settings and popups: ownership is visible behavior	11
10.1	Hidden, closed and dormant are not synonyms	11
10.2	Do not tell the native view the same thing every frame	11
10.3	Hover reliability needed integration repairs, not just geometry	11

10.4	The process is not the last document window	12
11	Tooling and diagnostics: trust what the app reports	12
11.1	Launch policy no longer belongs to screenshot capture	12
11.2	The documentation distinguishes history from current ownership	12
11.3	Capabilities are independent facts, cheaply presented	13
11.4	Build and bundled-tool identity are evidence, not guesswork	13
11.5	Tests guard boundaries; they do not certify the desktop	13
12	The final cleanup: smaller decisions, then smaller files	13
12.1	Delete duplication before adding another abstraction	13
12.2	Presentation code has recognisable homes	14
13	What the whole design enables	14
14	The bill, and the remaining debt	14
15	Evidence ledger and reproduction	15

2 The campaign at a glance

2.1 What changed in each commit

The commit subjects are not very descriptive, so this is the useful translation. Numbers below are exact physical lines across tracked Rust files, including comments, blanks and tests. Generated artifacts and reports are excluded.

Commit	Main contribution	Rust delta	app.rs
27f644c	Baseline, before the architecture campaign	—	14,130
45f1116	Launch policy, transport boundary, tool metadata, shared mutation leases	+481	14,130
53d315a	Completion transaction/view, PDF extraction, protocol codecs, save handoff and characterization	+1,722	13,844
bb693d7	Protected background saves and Git repository execution/codecs	+883	13,589
a17e6c9	Stable tab accessors, read-only Git views and cached capability reporting	+905	13,619
3579b98	Stable document store, synchronization, preview effects, bounded PDF/index work, shared observation, child lifecycle	+3,781	13,847
d3ca988	Integration audit repairs, hover/focus/link/Explorer follow-up and regression documentation	+917	13,980
8cd06ea	Leaf views plus build identity and window/Settings/folder repairs	+420	12,840
acafbcd	Shared navigation with explicit search-versus-source focus intent	+217	12,647
74cf297	Explorer input/action boundary and thinner active/parked save completion	+252	12,400
ff87e58	Preview admission/teardown cleanup, edge-case fixes, deduplication, icons and Settings controls	+241	11,318

This is a coverage map, not a claim that every line in a mixed commit implements its headline. For example, step 1 also carries user-visible window repairs. The detailed sections below identify those rather than charging them all to mechanical extraction.

2.2 What was already there

Before this range, the application already had tabs, stable numeric tab IDs, miTeX projection, a core document key/receipt model, undo, atomic file writing, preview recovery, native adapters, profiling support and focused Settings and tooltip modules. This campaign reused and strengthened those mechanisms.

It did not invent safe saving, asynchronous work, a type-safe core or multi-window support from nothing. The advance is chiefly better ownership, bounded work, more consistent integration and tests around the seams. The native unsafe-code allowlist and earlier ADRs are supporting context, not achievements newly introduced by these ten commits.

3 Documents: identity that survives movement

3.1 A tab is a record, not a slot

The cool bit. The storage model in `src/app/tabs.rs` separates identity from display order. These are selected actual fields:

```
records: BTreeMap<u64, TabRecord>,
order: Vec<u64>,
active: Option<u64>,
preview: Option<u64>,
```

What it replaced. The old model had parked records, a separately handled active document, positional active/preview indices and a parallel identity vector. Reordering meant preserving agreements among multiple representations. Stable IDs existed already; the refactor first made callers use them, then made the records themselves live under those IDs.

What we can do now. Reordering changes the order list instead of changing which document a delayed operation means. Active and designated-preview selection are independent. Empty workspace state has no open tab; a private empty presentation record serves the editor until a real document is created. Folding, saved editor-widget state, workspace and autosave metadata belong to each record for its lifetime.

What this enables later. Session restoration and richer tab organization can build on stable records instead of positional remapping. Cross-window tab moves would still need an explicit transfer of owner identity and services; this is a foundation for that work, not an implemented drag-between-windows feature.

Evidence and cost. Reorder, preview pinning, empty state, dirty close, selection/undo and delayed-result tests protect the model. Repeated activation is tested not to start extra TinyMist, index or workspace jobs. Widget state still has to be restored, and activation still rekeys document identity to reject stale replies. A stable ID does not imply zero-allocation tab switching. Sources: `a17e6c9`, `3579b98`; todos 184–186; docs/architecture-followup.md.

3.2 Saved bytes and visible edits have different identities

The cool bit. The document key answers which owner, document lifetime and revision a result belongs to. A tab ID answers which record to find. A close continuation token answers which pending action may proceed. These are distinct questions, and the code no longer pretends one integer can answer all three.

What it replaced. More paths depended on the active slot or an LSP-sized revision alone. Those can look unchanged after a tab switch or document replacement even when the result is no longer applicable.

Now. Background results can be routed to a parked record and still fail the document's own freshness checks. A late receipt does not become valid simply because the tab still exists. Likewise, a valid receipt cannot release an unrelated newer close request.

Later. The same separation makes other asynchronous edits easier to review: identify the destination, validate its lifetime/version, then authorize the specific continuation. It does not justify creating another identity hierarchy. Sources: `src/app/saves.rs`, `src/workflow.rs`, core document/workflow models.

4 Saving: responsiveness without relaxing safety

4.1 Immutable handoff, then protected background IO

The cool bit. The UI prepares a self-contained request and the worker consumes it. This actual handoff is small:

```
let input = SaveInput::new(request, expected, intent, continuation);
```

The existing worker then executes `crate::save_io::execute(input)`. The UI keeps routing metadata, not a second copy of the source. Successful IO returns the existing receipt and a durability outcome; failure does not fabricate a receipt.

What it replaced. The old calling path performed the atomic disk write itself. First, 53d315a introduced the immutable handoff while execution remained synchronous. Then bb693d7 moved the disk work behind the protected worker. That staging matters: the first change was preparation, not a latency improvement.

Now. Manual save, Save As, active autosave and parked autosave share one admission/completion adapter. A window has at most one admitted save and no unbounded pending save-payload queue. Further edits keep their autosave deadline. The user sees “Saving...” until completion rather than treating submission as durability. Completion wakes the owning window instead of adding an idle timer.

Later. Slow disk, injected failures and owner closure can be tested at the IO boundary without clicking through the application. A future alternate storage adapter would still need equivalent expected-state and durability semantics; remote saving is not provided by this refactor.

Evidence and cost. Channel-controlled tests hold a writer while UI dispatch remains runnable, reject duplicate admission and deliver outcomes after owner closure. The historical optimized 128-KiB/10-ms-delay probe measured median foreground dispatch at 21.792 ms synchronously versus 0.0169 ms in the background. Total completion was 21.792 versus 22.879 ms. That is responsiveness, not faster storage. Preparation/path normalization remains foreground work. See docs/background-saves.md for the environment and exact workload.

4.2 One resource lease, with the check inside it

The cool bit. App-owned operations on a canonical destination serialize, and the disk precondition is re-read while holding that lease. Two windows cannot both approve writes against one old baseline and then blindly overwrite each other through this path.

The small shared primitive in `src/resource_lock.rs` is:

```
pub(crate) fn with_resource<R>(
    path: &Path,
    operation: impl FnOnce() -> R,
) -> R
```

Its weakly held registry shares a mutex among canonical aliases without retaining every historical resource forever. Different resources do not share one global operation lock.

What it replaced. Checking before the mutation boundary left a gap where another app-owned operation could change the baseline. The refactor made lease sharing explicit and later placed save checking and persistence together inside it. Git mutations use the same principle with repository-root scope.

Now. A second save with stale expected bytes becomes a conflict. A confirmed overwrite is tied to the observation the user approved; a later change requires new confirmation. Autosave requires a known baseline.

Later and limits. This is a useful local transaction pattern for future mutations. It is not an OS lock and does not exclude arbitrary external editors or Git processes. There is no automatic conflict merge. Known-missing, known fingerprint and explicitly unchecked destinations remain distinct cases. Sources: 45f1116, bb693d7; `src/save_io.rs`, `src/private_workspace.rs`.

4.3 Completion is a gate, not a notification

The cool bit. Durable completion, current document identity, unchanged dirty state and the matching continuation all have to agree before save-before-close can proceed. Manual formatting is another conditional continuation, not a side effect of every successful write.

What it replaced. Active and parked completion branches duplicated policy. 74cf297 routes them through stable-tab lookup and the existing receipt gate. The later audit found a separate hole: a failed worker has no receipt, and its error handler could cancel a newer close request. PendingSave now retains the submitting continuation token for that error path too.

Now. An inactive saved tab cannot format or close the active tab. Uncertain durability may record persisted bytes but does not release close or formatting. A stale worker failure remains visible as a notice without cancelling a newer workflow. Save As rebinds workspace and synchronization identity only on the appropriate completion path. Formatting follows a successful unchanged manual save; autosave and the post-format write do not trigger a save/format loop.

Later and limits. This gives us a reusable review pattern for delayed actions, not a reason to make every result a generic transaction. Rekeying on activation can conservatively reject a receipt even when its disk write succeeded; rejecting an ambiguous result is preferable to falsely marking the current buffer clean. Sources: bb693d7, d3ca988, 74cf297, ff87e58; todos 187–190, 239, 242.

5 Tinymist: separate wire mechanics from document policy

5.1 A transport parser that knows nothing about the editor

The cool bit. `src/tinymist/transport.rs` deals only with bounded JSON-RPC framing over readers/writers. It neither owns a server process nor knows about completion popups. Its current bounds are explicit:

```
const MAX_HEADER_LINE_BYTES: usize = 8 * 1024;
const MAX_HEADER_BYTES: usize = 32 * 1024;
const MAX_MESSAGE_BYTES: usize = 32 * 1024 * 1024;
```

What it replaced. Framing lived inside the larger sidecar implementation. The extraction preserved the sidecar interface and thread model, rather than inserting another queue or copying payloads through a new transport object.

Now. Truncated headers/payloads, duplicate lengths and oversized frames can be exercised directly with in-memory streams. Bounds apply even to a header without a terminating newline. Framing changes need not involve native UI.

Later and limits. Different stream sources or protocol fixtures are easier to test. This is not a general LSP client framework and does not promise resilience to every behavior of an external server. Sources: 45f1116; todo 201.

5.2 Feature codecs borrow source instead of owning the world

The cool bit. `src/tinymist/protocol.rs` holds wire types and feature codecs. For example, a content change carries borrowed text:

```
pub(super) struct ContentChange<'a> {
    pub(super) text: &'a str,
}
```

What it replaced. DTOs and serialization logic were interleaved with session coordination. The new boundary keeps serialization separate without creating another source snapshot merely to construct a notification.

Now. Wire fixtures, fake-server tests and real-server integration can target the protocol contract. Completion items retain only fields the editor supports; commands attached to completion items are not automatically executed.

Later and limits. New supported LSP features have a clearer place for decoding and validation. Session still owns request IDs, pending calls, document versions, handshake phase and ordered sends. We explicitly decided not to split it again: that would duplicate ownership or add forwarding without an independent policy. The process supervisor stays separately reachable for forced termination when the writer blocks. Sources: 53d315a; todos 202–203.

5.3 Synchronization is a versioned plan; the app performs the IO

The cool bit. `src/tinymist_sync.rs` decides what needs opening, changing, closing or updating in a private backing file. Selected actual effect variants:

```
UpdateBacking(BackingTarget),
Open { generation: Generation, uri: String, version: i32 },
Change { generation: Generation, uri: String, version: i32 },
Close { generation: Generation, uri: String },
```

What it replaced. Startup, tab changes, active/preview document selection and backing ownership were more scattered across app orchestration. A parked-open path could use the active document's revision rather than its own.

Now. Inputs pair document key, URI, LSP version and canonical source. One coordinator tracks open URIs and backing lifetimes. The app remains responsible for writing files and calling the sidecar. Hover, formatting and completion replies are admitted against generation, current URI and revision before they may alter current state.

Later. Command-log tests can explore restart/switch/rename sequences without a real server. Another presentation mode can use the same canonical-source boundary rather than teaching transport about display coordinates.

Limits. Ordinary documents still bypass projection; no-consumer edit paths return before collecting canonical source. That preserves the zero-projection contract rather than charging every keystroke for miTeX. The CLI compiler still reads imported subfiles absent from editor overrides from disk. This is not yet a general TeX-engine or arbitrary Typst-compatible-binary architecture; todos 222–223 remain separate proposals. Sources: 3579b98, d3ca988; todos 191–193.

6 Preview: readiness is earned, resources are budgeted

6.1 A controller decides transitions, adapters execute them

The cool bit. PreviewController returns effects for restart/stop, refresh, raster scheduling and bounded repaint deadlines over the existing connection, recovery and content models. It does not acquire a sidecar or egui context.

What it replaced. More transition decisions lived in application event handling and rendering queries. The refactor did not introduce a second retry counter; it consolidated use of the existing recovery model.

Now. Duplicate terminal events do not spend two attempts. Five consecutive failures lead through four delayed retries to fallback. Requested backend, effective backend, native readiness and available canonical output are separate facts exposed in a read-only status snapshot. Rendering consumes that snapshot instead of making another recovery decision.

Later. Recovery can be tested by advancing supplied time and inspecting effects, without sleeps or launching a server. Additional recovery cases can be added at the policy boundary while native objects stay on their required thread.

Limits. The app still sequences several distinct owners. Two generation checks are not automatically duplicates: synchronization and connection recovery can have different states. Sources: 3579b98; todos 194–196.

6.2 An endpoint is not ready just because a message says so

The cool bit. The later preview_ready admission method verifies the attempt and initialized connection before accepting the endpoint. Selected actual code:

```
if !self.tinymist_preview_enabled
  || !self.recovery.accepts(generation)
  || !self.connection.is_ready_for(generation)
{
  return Ok(false);
}
let endpoint = url::Url::parse(url).map_err(|_| "Invalid preview endpoint")?;
if !self.connection.connect(generation, endpoint) {
  return Ok(false);
}
self.recovery.recovered(generation);
```

What it replaced. Recovery could be marked successful before URL parsing and connection admission. Repeated bad endpoints could therefore reset the very failure budget intended to reach fallback.

Now. Invalid endpoints use the normal retry path; stale, uninitialized, suspended and LSP-only messages cannot request embedding. Reusing the same URL after server replacement does not make a retained native surface

evidence of current readiness. Native teardown also has one implementation that clears the view and its related cached identity together.

Later and evidence. We can add readiness scenarios without duplicating their ordering in every app branch. Tests cover five malformed-endpoint failures, same-URL replacement, LSP-only readiness and suspension. A separate regression caught suspended enabled-state repeatedly requesting restart; 100 later checks now emit no effects. This is correctness, not a measured FPS claim. Source: ff87e58; todo 240; docs/preview-ownership.md.

6.3 PDF decoding is no longer a compiler responsibility

The cool bit. `src/pdf.rs` supplies catalog/raster/link operations to compiled documents, opened PDFs and thumbnails. The compiler produces the artifact; presentation chooses what to render.

What it replaced. Poppler command construction, decoding and links lived with compiler machinery, so consumers depended on the wrong conceptual owner.

Now. One implementation handles numeric page order, raster modes and links. The canonical PDF bytes remain the export source; changing displayed scale or appearance does not degrade export quality. Reader shutdown belongs in shared process support rather than another PDF-specific copy.

Later and limits. New PDF consumers can reuse the service without pretending to compile a document. This does not remove Poppler dependencies or make PDF rendering free. The initial extraction added no worker or cache on its own. Source: 53d315a; todo 197.

6.4 Keep the page map; rent the pixels

The cool bit. All-page dimensions and links are lightweight metadata. Decoded pixels and uploaded textures are separately resident, keyed by artifact, page, DPI and appearance. The fallback view requests its visible range, prefetches a neighbor on each side and caps a request at twelve pages.

What it replaced. An all-page decode loop built a collection containing every page's RGBA buffer. Moving that loop to a new file would not have solved its resource scaling; demand-driven residency does.

Now. Offscreen pages need geometry, not image widgets and resident pixels. Superseded rendering is cooperatively cancelled, stale artifact/appearance results are rejected, and valid old content remains displayed until replacement content is ready. Typst and opened-asset surfaces retain independent workers because both can be visible simultaneously.

Later. Zoom/prefetch policy can evolve against a bounded page-demand interface. We could make residency pressure adaptive, but that would need measurements and new policy; the current implementation uses fixed budgets.

Evidence and limits. Tests cover page ordering, cancellation, prefetch bounds, replacement retention and stale results. The audit repaired child-process failure handling and page-number admission: receiving some output is not proof a render succeeded, and missing pages must not shift later page identities. Sources: 3579b98, d3ca988; todos 198–199, 224.

6.5 Multiple windows share a resource budget

The cool bit. `src/pdf_residency.rs` accounts decoded pixels and estimated texture bytes process-wide:

```
pub(crate) const DECODED_PIXEL_BUDGET: usize = 96 * 1024 * 1024;
pub(crate) const TEXTURE_BUDGET: usize = 192 * 1024 * 1024;
```

Eviction invalidates a lease and wakes its owner. The actual texture is dropped on that owner's UI thread. Nonvisible resources are preferred eviction victims; an oversized page may be admitted alone instead of becoming impossible to view.

What it replaced. All-page retention, and the risk of reasoning about each window's memory in isolation. Four windows cannot each treat the full budget as their private allowance through this accounting path.

Now. Deterministic tests can assert bounded residency across 1/20/100-page documents and multiple owners. The historical synthetic 8-MiB-per-resource comparison modeled 1,600 MiB for a 100-page all-page collection versus 24 MiB after the bounded workload.

Later and limits. This is a useful home for memory-pressure decisions. The figures are not measured OS/GPU residency and the budgets are not hard process caps: transient allocations, delayed destruction and WebKit are outside that claim. Sources: 3579b98, d3ca988; todo 200; architecture follow-up evidence.

7 Workspace and indexing: stop doing work nobody needs

7.1 Two workers, not one obsolete thread per request

The cool bit. The project index has a specialized process-wide runner:

```
const CONCURRENCY: usize = 2;
const MAX_PENDING_BYTES: usize = 64 * 1024 * 1024;
const MAX_REQUEST_BYTES: usize = 32 * 1024 * 1024;
```

What it replaced. Dropping the previous receiver ignored old results but did not stop their computation. Rapid requests could leave many obsolete jobs alive.

Now. Pending payloads are replaced by owner/source key, active work receives cancellation, and checks occur between files and every 256 syntax nodes. Results carry request/owner identity; only the requesting viewport is woken. The old 50-ms UI polling repaint is gone. This runner cannot execute mutations or own a long-lived protocol session.

Later. We have one place to inspect queue behavior and tune fairness under many windows. There is no justification yet for a generic priority scheduler.

Evidence and cost. The historical four-window, 100-request synthetic probe used two workers instead of 100 spawned threads and completed the latest results in about 11.34 ms versus 193.15 ms for the compared old model. That establishes this workload, not universal typing latency. Byte admission and cancellation tests are the stable contract. Sources: 3579b98; todos 204–206.

7.2 Refusing new work must not destroy good old work

The cool bit. Queue admission now computes the projected byte use before cancelling or replacing existing work.

What it replaced. The first bounded-runner implementation could cancel a usable old request and then discover the replacement did not fit. A bound that protects memory but loses the last valid update is not a complete policy.

Now. Rejected same-key and owner replacements preserve queued/active work. The client's latest-request bookkeeping changes only after successful admission.

Later. This is the pattern to reuse when adding any bounded replaceable queue: validate the whole replacement before discarding what it replaces. It is a more valuable lesson than the queue's specific data structure. Source: d3ca988; todo 224; src/index_jobs.rs admission regressions.

7.3 One workspace tree, many local Explorers

The cool bit. Windows on the same canonical root share immutable Arc<WorkspaceSnapshot> data and one observation service. Expansion, search, selection and section layout remain window-local.

What it replaced. Redundant per-window scanning and routine full-content checks. Sharing data does not mean sharing which folder you have expanded.

Now. Structural notifications coalesce after a quiet period; content/metadata events need not rescan the tree. Own-output and access events are filtered to avoid feedback loops. A conservative verification scan catches missed events. Active-file checks use metadata before content hashing. Closing one subscriber does not invalidate another's tree.

Later. More windows can reuse observation work without synchronizing their presentation. Workspace, language index and Git remain separate models; a single all-purpose filesystem model would blur their different guarantees.

Evidence and limits. Tests demonstrate pointer-identical snapshots and one initial scan for two windows. Audit repairs added subscription identity so old root events cannot be applied after switching, and hardened

owner bookkeeping and stopped-observer recovery. Native event delivery is still imperfect, which is why the verification fallback remains. Sources: 3579b98, d3ca988; todos 208–209, 224.

7.4 An index can admit what it does not know

The cool bit. IndexCompleteness records traversal caps and unreadable files, alongside the existing unresolved dynamic dependency expressions. This is a literal-source index, not a second Typst evaluator.

What it replaced. Results could appear complete even when traversal stopped or a dependency required evaluation. The first attempt at exposing that state also produced unhelpful persistent technical warnings.

Now. Limits have a model-level representation and can be explained without reformatting warnings every Explorer frame. Follow-up removed persistent jargon from ordinary rows. Missing-path normalization is cached within an analysis; the measured missing-import regression was repaired without introducing a stale persistent filesystem cache.

Later and limits. We can build a useful explanation around the relevant file, line and expression. That is more actionable than a count of “dynamic dependencies.” The model does not imply downloaded packages are indexed or that dynamic imports are resolved. Sources: 53d315a, d3ca988; todo 207 and docs/architecture-performance.md.

8 Editing and views: make mutations deliberate

8.1 Completion is one consumable edit

The cool bit. CompletionTransaction owns a validated edit and full identity; commit consumes it. Selected actual signature and gate:

```
pub(crate) fn commit<C>(  
    self,  
    document: &mut tiptotyp::mitex_document::Document<C>,  
    before: C,  
) -> Result<Range<usize>, String> {  
    if document.key() != self.key {  
        return Err("the completion belongs to an outdated document".to_owned());  
    }  
    // Projection preflight and one document edit follow.
```

What it replaced. Completion preparation/application lived closer to app editing orchestration, with weaker emphasis on full identity and one undo transaction. The refactor reuses core range validation rather than another coordinate implementation.

Now. Main/additional edits, snippet expansion, Unicode and CRLF are validated; canonical and display coordinates are explicit. miTeX canonical changes pass projection preflight. Commit edits the document once and returns selection, rather than partially applying an invalid batch. Consuming self makes accidental reuse of the same transaction impossible through this API.

Later and limits. Other structured edits can follow the same prepare/validate/ commit pattern. They should not be forced into a completion-specific type. Typing and IME remain with TextEdit. Sources: 53d315a; todo 210.

8.2 The completion popup borrows its payload

The cool bit. The popup takes borrowed items/source and returns selection, acceptance or dismissal. Font painting is supplied by its adapter. It cannot ask a catalog to scan or start a language-server request.

What it replaced. The handoff cloned completion items, including potentially large documentation, and mixed more application access into presentation.

Now. Merely hovering an unchanged selection does not repeatedly emit selection actions. A historical 256-item fixture measured 1,135,250 requested allocation bytes per cloned handoff versus zero for the borrowed handoff.

Later and limits. Presentation can evolve independently of completion acceptance. Zero-copy handoff does not mean allocation-free popup rendering: labels, layout, tooltip content and font previews still have costs. Borrowed-loop timing was below meaningful per-call resolution, so no huge speedup ratio is claimed. Sources: 53d315a; todo 211; tests/completion_payload_cost.rs.

8.3 Search selection and source navigation are different intents

The cool bit. EditorSelection distinguishes Search from Focus. Both can move the selection, but only source navigation should take editing focus away from the Find field.

What it replaced. An undifferentiated pending range lost the caller's intent. With Find open, a source jump could move the caret while leaving keyboard focus in Find. Other navigation paths had their own current-file checks.

Now. Problems, index/file links and preview/LSP navigation share the handoff while respecting their coordinate domains. A later audit also unified current source identity so an untitled private backing path does not open as a different document. Focus order remains native parent, viewport, then editor.

Later and limits. New navigation sources can request the same explicit intent. Root/secondary egui tests cover selection, one-shot focus and word/line keys. They do not prove macOS WebKit handed real keyboard events back to TextEdit. Todo 237 intentionally remains open for that native acceptance. Sources: acafbcd, ff87e58; src/app/navigation.rs.

8.4 Explorer painting returns requests, not side effects

The cool bit. explorer_view::show receives borrowed tree/index/status data and presentation state. Its output includes requests such as:

```
pub open: Option<PathBuf>,  
pub index_target: Option<(PathBuf, usize)>,  
pub context_menu: Option<ContextMenu>,
```

What it replaced. show_workspace combined a much larger body of Explorer layout, interaction and app effects. Step 1 moved leaf helpers; step 3 narrowed the actual rendering boundary. Those are different levels of improvement.

Now. The app applies requests after rendering. Search focus and Git reveal live in ExplorerPanelState; tree IDs, clipping, filters and section persistence are preserved. Tests can render the view without constructing EditorApp or launching a watcher. The later cleanup removed parallel output locals, an unnecessary generic result and repeated empty-state branches.

Later and limits. Explorer presentation can change with less risk of initiating a scan or navigation during painting. It still receives a Git paint callback, and owner-scoped hover/drop helpers remain presentation dependencies. This is a useful narrow boundary, not an independent application framework. Sources: 8cd06ea, 74cf297, ff87e58; todos 236, 238, 242.

9 Git: transactions below, actions above

9.1 Repository execution has its own home

The cool bit. A borrowed Repository handle groups subprocess discovery, bounded execution, snapshots and index operations. Diff codecs and checked hunk construction have dedicated modules without UI imports.

What it replaced. Command/codecs logic was interwoven with panel/editor code. The extraction did not introduce a repository daemon or move workers merely to make a new service abstraction.

Now. Commands still use argument arrays, literal pathspecs, bounded output and timeouts. Disposable-repository tests cover quoted/Unicode paths, new files, CRLF, missing final newlines and exact index bytes. A discovered nested-workspace routing bug was fixed: result destination retains the requesting workspace, while snapshot metadata retains the resolved repository root.

Later and limits. New repository actions can reuse the execution boundary without importing UI state. Process execution still costs time, and changing where it lives does not itself speed up Git. Sources: bb693d7; todo 212.

9.2 Stage/unstage and panel mutations share a transaction boundary

The cool bit. Hunk index mutations use the repository-root lease and revalidate their baseline inside it, just like panel operations. Checked revert produces a proposed editor change applied through one undo transaction.

What it replaced. Independent paths could mutate one Git index based on stale observations or fail to coordinate with panel operations. The lease fix was done before waiting for the visual/service extraction.

Now. Conflicting state is rejected; unrelated staged changes are preserved. The audit later made hunk arithmetic checked and validated range/body agreement before constructing edits, so malformed diff text is an error rather than a panic or mis-sized replacement.

Later and limits. More hunk operations can use the same guarded boundary. External Git processes do not participate in the app's mutex; that limitation must remain explicit. Sources: 45f1116, bb693d7, d3ca988; todos 213, 224.

9.3 Rendering a Fetch button cannot fetch

The cool bit. The Git panel and gutter/popup consume snapshots and return typed actions. A semantic click test demonstrates the renderer emits an action; the application later admits the operation.

What it replaced. Presentation was more tightly coupled to repository access and mutation admission. Hunk controls also occupied more vertical space.

Now. Compact hunk actions sit on the title row with full accessible labels and shortcut hints. Commit text is cloned only for an edit or Commit request. Diff strings are shared and the bounded galley cache stays with view state, keyed by content/style/font identity.

Later and limits. Git layout can be tested without a repository or subprocess. This is a safer place for UI experimentation, not a claim of zero rendering cost. Sources: a17e6c9; todo 214; src/git/view.rs, src/git/editor/view.rs.

10 Windows, Settings and popups: ownership is visible behavior

10.1 Hidden, closed and dormant are not synonyms

The cool bit. Child-view state distinguishes Visible, TemporarilyHidden, DurablyClosed and DormantHosted. A durable close advances generation so a late deferred callback becomes inert; temporary hide can retain the surface.

What it replaced. Treating every absence as equivalent either destroys useful retained state or leaves callbacks/resources belonging to a dead owner active.

Now. Dormant document owners close transient children but retain explicitly hosted surfaces such as Settings. Durable close disposes owned font-preview slots. Tests check reopen, stale callbacks and live cache counts returning to baseline; hidden pickers can retain their bounded sample cache.

Later and limits. Further child surfaces have an explicit lifecycle to join. Not every hidden surface should be destroyed, and native composition still requires native evidence. Source: 3579b98; todos 215–216.

10.2 Do not tell the native view the same thing every frame

The cool bit. Applied native state is diffed before setting background, bounds and visibility. Recreation and teardown invalidate that cache.

What it replaced. The adapter issued all three setters on unchanged frames. The recorded deterministic model went from 300 calls over 100 stable frames to zero after creation. A bounds change needs only the bounds update.

Now. Stable frames avoid redundant native calls without moving AppKit/WebKit work to arbitrary threads. The later shared `discard_webview` prevents individual teardown paths from forgetting a cache field.

Later and limits. Additional native properties can use the same compare/apply discipline if traces justify it. The setter count is not an FPS measurement. Recorded viewport captures and geometry traces are not proof of composed native window pixels. Sources: 3579b98, ff87e58; todos 217, 240.

10.3 Hover reliability needed integration repairs, not just geometry

The cool bit. The repairs treat handoff geometry, timer ownership and native child lifetime as one interaction sequence. The safe region points from the captured source position to the facing edge of the popup; other dismissal paths must respect the handoff too.

What it replaced. A triangle test could pass while owner cleanup closed the popup anyway. Non-hovered controls could erase another control's timer. A delayed frame could be misread as evidence that the pointer left. Asset hover also repeated syntax work across pointer frames.

Now. Repairs preserve shared timer ownership, avoid resetting merely because a frame was late, reuse prepared syntax, retain the native handoff route and bound external-link launching so repeated slow clicks do not fan out unchecked. Explorer restoration similarly rejects unusable constrained sizes instead of persisting them as the preferred width. Index limitation wording is kept out of persistent ordinary rows.

Later and limits. These are examples of sequence-level regression tests we can reuse for other popups. They are also why we must not call every earlier fix successful: the user found repeated failures despite passing component tests. Current native hover/focus/latency acceptance is not established by this document. Source: d3ca988 and later navigation fixes; todos 224–227, 237.

10.4 The process is not the last document window

The cool bit. The shell handles process-level creation and Settings separately from document-local commands. With no document window, New Window/Open in New Window/Settings remain meaningful; ordinary New/Open/Copy are not treated as operations on a hidden document.

What it replaced or repaired. The campaign inherited retained-root machinery but exposed no-document and multi-window failures. Follow-up routes every Settings request to the one root-owned Settings surface, allows folder opening into an empty workspace, and preserves the pinned preview when New creates an editor tab. Profiling's no-window transition was moved to the same end-of-frame retirement boundary to avoid detaching the macOS CGL view prematurely.

Now. Two document windows do not each create their own Settings window. No-document Settings does not infer “non-Typst” merely from unavailable preview. The shell tests command routing and dormant/reopen state without requiring an interactive picker.

Later and limits. Process-level features have a natural home in AppShell, not every EditorApp. This does not authorize moving each native window's UI to an arbitrary thread, or prove every native close/reopen sequence by headless tests alone. Sources: 8cd06ea; regression execution follow-up in `todo.typ`.

11 Tooling and diagnostics: trust what the app reports

11.1 Launch policy no longer belongs to screenshot capture

The cool bit. `LaunchOptions` and `LaunchMode` describe normal, deterministic capture and profiling starts before editor state is constructed.

What it replaced. Ordinary startup argument/persistence policy lived under screenshot ownership. That confused a process-level concern with one consumer.

Now. Parsing is testable with supplied arguments/environment; deterministic fixtures cannot silently overwrite normal settings. Profiling storage is session-owned. The process wrapper still reads its actual environment/cwd; the parser itself does not discover tools or inspect workspace contents.

Later and limits. More launch modes can be reasoned about explicitly, but ordinary startup should not become a general configuration framework. Source: 45f1116; todo 218; `src/launch.rs`.

11.2 The documentation distinguishes history from current ownership

The cool bit. ADR 0002 retains the original decision rationale but adds a superseding current-state amendment. The public `miTeX` module comments no longer describe an already-enabled feature as a future foundation.

What it replaced. Old comments could lead a reader to think Settings replaced the workspace, Tinyminst ownership followed an outdated model, or interactive preview necessarily ran a parallel CLI compile on every edit.

Now. The maintained explanation identifies per-window services, tab-owned documents, independent Settings presentation, conditional CLI artifact production and canonical export bytes. It also states the unsaved-import limitation rather than implying a complete compiler overlay.

Later and limits. This makes onboarding and future design reviews less likely to repeat already-resolved questions. Documents still age: the singleton Settings repair and current CI matrix must be read alongside earlier ADR descriptions. The code and later amendments win when they disagree. Source: `todo 221; docs/architecture/0002-interactive-editor.md, src/lib.rs`.

11.3 Capabilities are independent facts, cheaply presented

The cool bit. A cached snapshot separates editing, LSP, interactive preview, PDF generation, rasterization and link extraction.

What it replaced. Coarse availability reporting could hide a working LSP because the compiler was missing, or conflate Poppler's distinct executables.

Now. Optional executable discovery happens on session creation and explicit Refresh tools, not during Settings rendering. Tool preference changes invalidate derived state without probing unrelated tools again. Audit fixes require both inspection and raster tools where rasterization actually needs both.

Later and limits. Better actionable tool diagnostics can consume the snapshot without adding filesystem/process work to every frame. Discovery establishes availability, not that every invocation or project will succeed. Sources: `a17e6c9, d3ca988; todo 219; src/capabilities.rs`.

11.4 Build and bundled-tool identity are evidence, not guesswork

The cool bit. Runtime bundled-tool versions are generated/validated against `toolchain/manifest.tsv`. A separate compiled-in app build identity can be shown without running Git or reading disk on every launch/status frame.

What it replaced. Independently maintained tool-version constants could drift from packaging; stale app bundles were hard to distinguish from a rebuilt binary.

Now. Build checks reject inconsistent tool metadata, and `src/build_info.rs` combines package version with the compiled build ID. We can name the executable we actually tested instead of trusting its icon or app name.

Later and limits. Bug reports and profiling metadata can include reproducible identity. This does not prevent the OS or user from launching an old bundle; it makes that mismatch diagnosable. Sources: `45f1116, 8cd06ea; todo 220`.

11.5 Tests guard boundaries; they do not certify the desktop

The cool bit. Architecture tests reject application/service access from focused views, direct tab-storage access from callers, PDF decoding returning to the compiler, and platform effects leaking into the pure core. Semantic tests can exercise real controls while fault-injection tests force delayed/out-of-order completion paths.

What it replaced. Some obligations were conventions rather than executable checks, and the first response to regressions placed too much manual testing burden on the user. Follow-up reports distinguish developer-executable checks from the small manual acceptance remainder.

Now. CI is macOS-first: Windows was removed; Linux remains nonblocking portability signal. A separate macOS real-tool job fetches pinned sidecars and runs explicitly requested integration tests. The older ADR's three-platform matrix description is historical, not the current workflow.

Later and limits. Better targeted native automation can complement these layers. Source-string guards are useful but not proofs of semantics. Passing 1,074 tests is evidence for their cases, not proof against every focus, scrolling or timing regression. Sources: `the commit range's tests, 3579b98 CI change, tests/architecture_boundaries.rs, .github/workflows/checks.yml`.

12 The final cleanup: smaller decisions, then smaller files

12.1 Delete duplication before adding another abstraction

The cool bit. The most recent cleanup did not create a framework. It shared document-open setup while preserving destination/title/filter differences, shared raster zoom effects between keys and buttons, removed an empty title forwarder and reused existing menu-availability defaults.

What it replaced. Two dialog paths repeated the same filters, directory, parent/admission and request construction. Raster zoom repeated the same clamp and fit-width transition. Explorer also had parallel output locals and repeated empty-state branches.

Now. The dialog and zoom pass removes 60 production lines; its 39 test lines leave a net 21-line Rust reduction. The preceding audit cleanup removes 74 production lines but adds 89 test lines. Both figures are recorded honestly rather than hiding test growth or counting moved code as deleted.

Later and limits. This is the right shape for further consolidation: identify the repeated implementation, preserve meaningful differences, delete the copies. It is not a credible promise of thousands of easy deletions. Source: ff87e58; todos 242–243; postpostmortem.typ section 4.

12.2 Presentation code has recognisable homes

The cool bit. Explorer helpers, package browsing and popup geometry first moved to their own modules. The later split puts vector icon kinds/painting/ geometry in `app/icons.rs` and reusable font/weight/syntax/tool-status controls in `app/settings_controls.rs`. The new Settings callers import those controls directly from their sibling module.

What it replaced. Cohesive leaf presentation code was buried among app-level orchestration. We did not move it into an indiscriminate `helpers.rs` or create another controller merely to shorten the root file.

Now. You can find icon geometry without scrolling past compiler and save handling, and reuse a Settings control without granting it `EditorApp`. The moved bodies were compared against their originals; differences were visibility, imports and formatting, not drawing or state algorithms.

Later and limits. These modules are easier to review and test in isolation. The last split alone reduced `app.rs` by 961 lines and grew total Rust by 46. It is organization, not performance work or a new ownership model for the app's remaining 128 field declarations. Sources: 8cd06ea, ff87e58; todos 236, 244.

13 What the whole design enables

The repeated theme is **identity, ownership and bounded cost**. A tab identifies the record; a document key identifies its lifetime/version; a continuation identifies the pending action; a generation identifies the server attempt; a resource lease identifies which app-owned mutations must serialize.

That separation lets us ask specific questions in tests: did this result target the right owner, was it still current, was the side effect admitted once, and did it stay within the intended work/resource bound? Those questions are much more useful than “did the function return successfully?”

The plausible next opportunities are correspondingly concrete:

- Session/tab restoration using stable records, with explicit persistence and owner-transfer rules rather than serialized UI internals.
- Better scheduling and memory-pressure policy using the existing bounded index runner and PDF residency model, only after matched workloads justify changes.
- More structured editor operations using validated, one-undo transactions and the existing canonical/display distinction.
- Improved native integration tests around source-to-popup and preview-to-editor focus, using current-build identity and real input delivery.
- Further deletion-first consolidation of repeated adapter decisions, while leaving distinct safety gates intact.

None requires a new universal event bus, generic executor or a rewrite of the UI toolkit. A future TeX engine or alternative compatible compiler is a separate product/architecture investigation, not a feature these commits already deliver.

14 The bill, and the remaining debt

The full interval grows from **85,253 to 95,072 Rust lines: +9,819**. `app.rs` falls from **14,130 to 11,318: -2,812**. Using conventionally named test files and only the actual top-level inline test blocks, identified test lines grow from 27,819 to 32,864 (+5,045); the remainder grows from 57,434 to 62,208 (+4,774). The latter is an estimate containing comments, tooling and scattered test helpers, not a compiler-derived count of executable production statements.

The earlier postmortem’s inline-test suffix method overcounted tests when production code followed the test module. The corrected method used here stops at the module’s own unindented closing brace, following this repository’s rustfmt layout. Exact total counts do not depend on that approximation.

Some of the growth pays for real guarantees: bounded execution/residency, background save safety, identity checks and regression coverage. Some pays for interfaces and repaired integration mistakes. It is not all tests, and smaller files do not prove lower RAM or CPU consumption.

The central coordinator is still too broad. Many child modules still implement methods on EditorApp, which keeps app-wide access even after relocation. Native focus acceptance remains open. Repeated user-discovered failures show why a plausible fix plus unit tests cannot automatically be called done. The correct stance is to keep the demonstrated safety/resource gains and continue reducing duplicated decisions, not declare the entire architecture finished.

15 Evidence ledger and reproduction

This document introduces no runtime code and reruns no application benchmark. The preceding implementation validation recorded 1,074 full-suite passes, 16 opt-in tests ignored, 13 xtask passes, formatting and strict Clippy. The historical performance probes cited above used Apple M2 Max, arm64 macOS 14.6.1 and Rust 1.96.0; consult their recorded metadata before comparing a new run. Native steady-state measurements in docs/architecture-performance.md precede much of the campaign and do not certify the final endpoint.

Two more acceptance gaps deserve explicit names, rather than disappearing into a general caveat. Todo 229 leaves native hover profiling unresolved: that scenario timed out before readiness while other scenarios completed, and the capture/event path must be diagnosed before calling it an application hang. Todo 234 leaves the transient oddly shaped Settings-window flash unresolved; singleton ownership and steady-state observations do not prove a brief flash is gone. regression-results.typ records actual native close/reopen, singleton Settings, folder-launch and pinned-preview observations, and the picker-selection automation timeout. These are useful partial observations, not all-or-nothing desktop certification. Todo 235 records developer-owned checks taken over from the manual checklist instead of assigning another large checklist to the user.

The retained sources for each layer are:

- docs/architecture-performance.md: matched early workloads, missing-import regression, native sampling limitations and commands.
- docs/architecture-followup.md: tab/index/PDF/completion probes, shared observation, synchronization, Git views and capability contracts. Some “next work” paragraphs describe intermediate stages; later sections supersede them.
- docs/background-saves.md: save ownership, ordering tests and foreground dispatch versus total-time measurements.
- docs/preview-ownership.md: readiness/visibility/teardown owners and the step-5 audit. Later todo 242 records two additional bugs that audit missed.
- todo.typ 184–221: architecture issue ledger; 224–244: repairs, extraction, acceptance limits and local line accounting. Items 222–223 and 237 are not silently promoted to completed features by this walkthrough.
- postmortem.typ and postpostmortem.typ: campaign costs, correction of the counting method and deletion-first recommendations.
- The ten commits and current source modules named in each section: authoritative implementation evidence when an earlier document describes an intermediate state.

To reproduce the scope and exact counts without checking out old code:

```
git log --reverse --oneline 27f644c..ff87e58
git diff --numstat 27f644c ff87e58 -- '*.rs'
git show 27f644c:src/app.rs | wc -l
git show ff87e58:src/app.rs | wc -l
```

For repository totals, enumerate each revision with `git ls-tree -r --name-only REF`, select .rs paths, and sum physical lines from `git show REF:PATH`. Do not include PDFs, screenshots or audit logs in Rust growth. Small excerpt omissions in this report are explicitly marked; selected field/variant lists are not complete compilable definitions.

The part worth showing off: the app now has more places where a result must prove it belongs, a mutation must earn permission, and background work must fit a budget. Those are useful architectural achievements. The next job is to keep them while making the surrounding integration smaller and more dependable.